 Run in Google Colab  View on Github  View raw on Github

Module 10: Logscale

In this module, we will learn why we want to use log scale for some types of data and strategies for using log scale.

```
import matplotlib.pyplot as plt
import numpy as np
import vega_datasets
```

Ratio and logarithm

If you use linear scale to visualize ratios, it can be quite misleading. As learned in the class, ratio values larger than 1 can vary between 1 and infinite, while ratio values smaller than 1 can vary only between 0 and 1. For instance, the ratios of 100:1 (100/1) or 1000:1 (1000/1) are represented as 100 and 1000. The corresponding distances from 1:1 (1) are 99 and 999, respectively. On the other hand, the ratios of 1:100 (1/100) or 1:1000 (1/1000) are represented as 0.01 and 0.001. The corresponding distances from 1:1 (1) are 0.99 and 0.999, respectively. In other words, there is no symmetry between symmetric ratios!

You can watch my video Why you should use logarithmic scale when visualizing ratios.

To see this clearly, let's first create some ratios.

```
x = np.array([1,    1,   1,  1, 10, 100, 1000])
y = np.array([1000, 100, 10, 1, 1,  1,   1   ])
ratio = x/y
print(ratio)
```

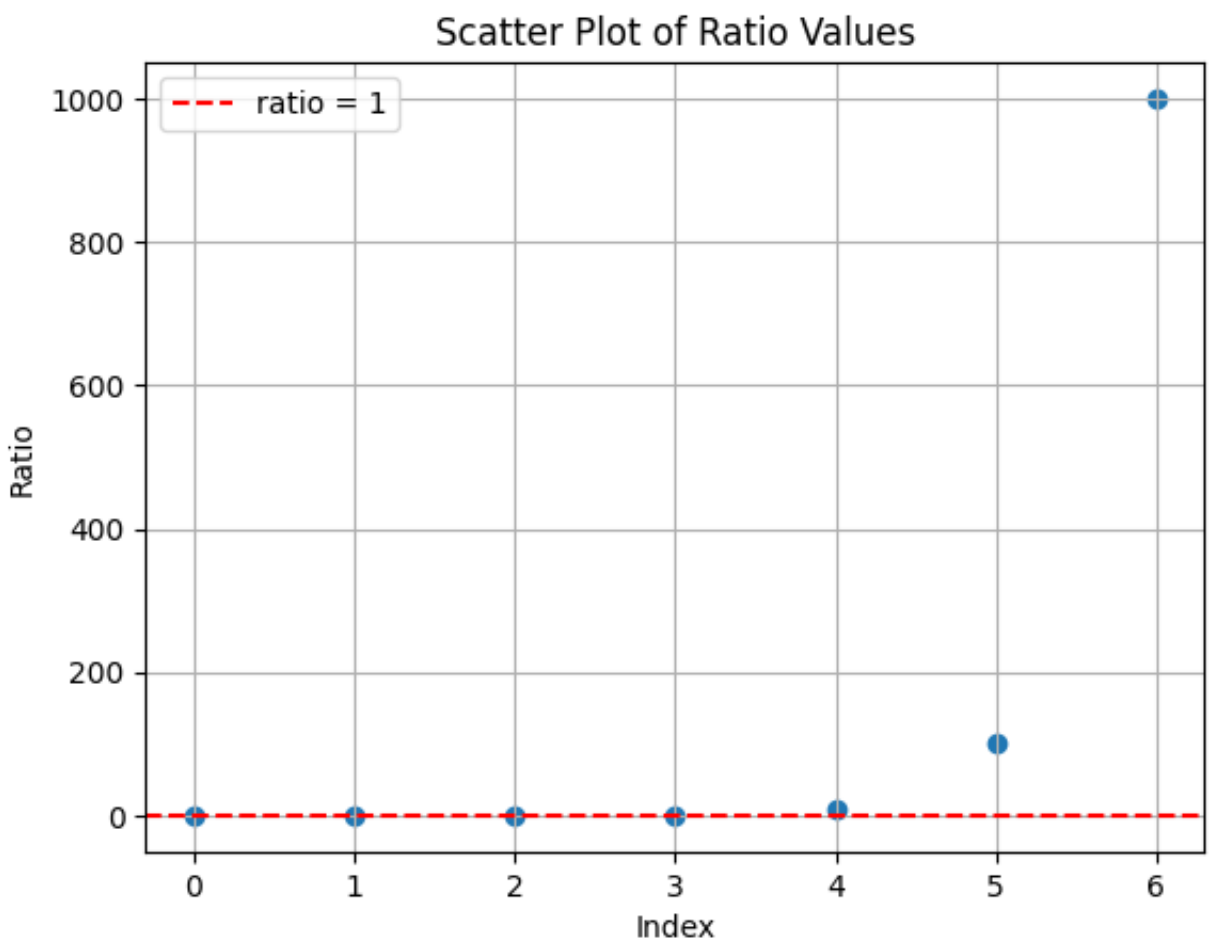
```
[1.e-03 1.e-02 1.e-01 1.e+00 1.e+01 1.e+02 1.e+03]
```

Q: Plot on the linear scale using the `scatter()` function. Also draw a horizontal line at ratio=1 for a reference. The x-axis will be simply the data ID that refers to each ratio data point. Y-axis will be the ratio values.

```
X = np.arange(len(ratio))

# YOUR SOLUTION HERE

plt.scatter(X, ratio)
plt.axhline(y=1, color='red', linestyle='--', label='ratio = 1')
plt.xlabel("Index")
plt.ylabel("Ratio")
plt.title("Scatter Plot of Ratio Values")
plt.legend()
plt.grid(True)
plt.show()
```



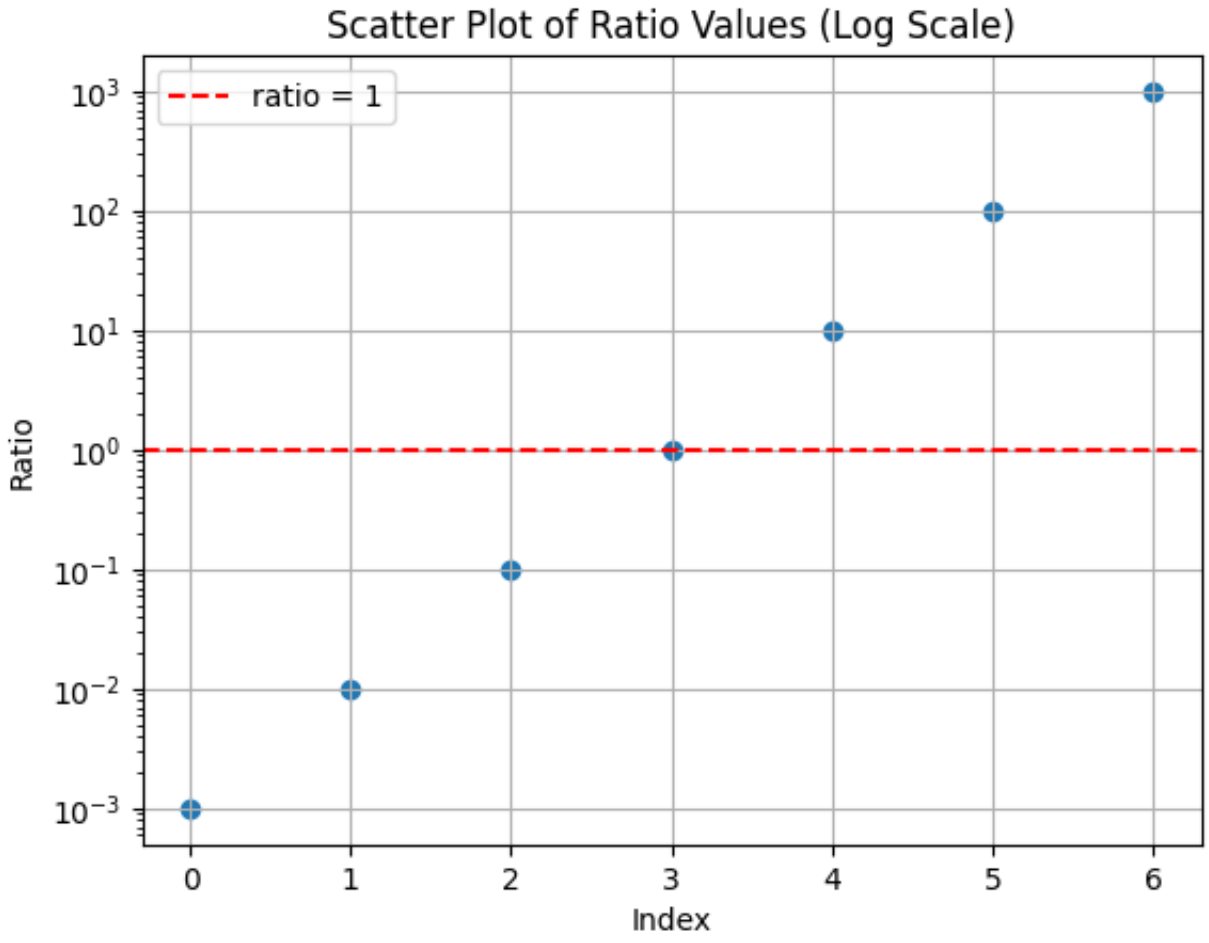
Q: Is this a good visualization of the ratio data? Why? Why not? Explain.

No, this is not a good visualization because the ratio values vary across several orders of magnitude. The large values dominate the chart and compress the smaller values near zero, making it hard to see most of the data points. A log-scale y-axis would better capture the symmetry and spread of the data.

YOUR SOLUTION HERE

Q: Can you fix it?

```
# YOUR SOLUTION HERE.
plt.scatter(X, ratio)
plt.axhline(y=1, color='red', linestyle='--', label='ratio = 1')
plt.xlabel("Index")
plt.ylabel("Ratio")
plt.yscale("log")  # <- Fix: use log scale for y-axis
plt.title("Scatter Plot of Ratio Values (Log Scale)")
plt.legend()
plt.grid(True)
plt.show()
```



Log-binning

One way to draw a histogram in log-scale, with a broadly distributed data, is by using log-binning.

Let's first see what happens if we do not use the log scale for a dataset with a heavy tail.

Q: Load the movie dataset from `vega_datasets` and remove the NaN rows based on the following columns: `IMDB Rating`, `IMDB Votes`, `Worldwide_Gross`, `Rotten Tomatoes Rating`.

```
# YOUR SOLUTION HERE
from vega_datasets import data

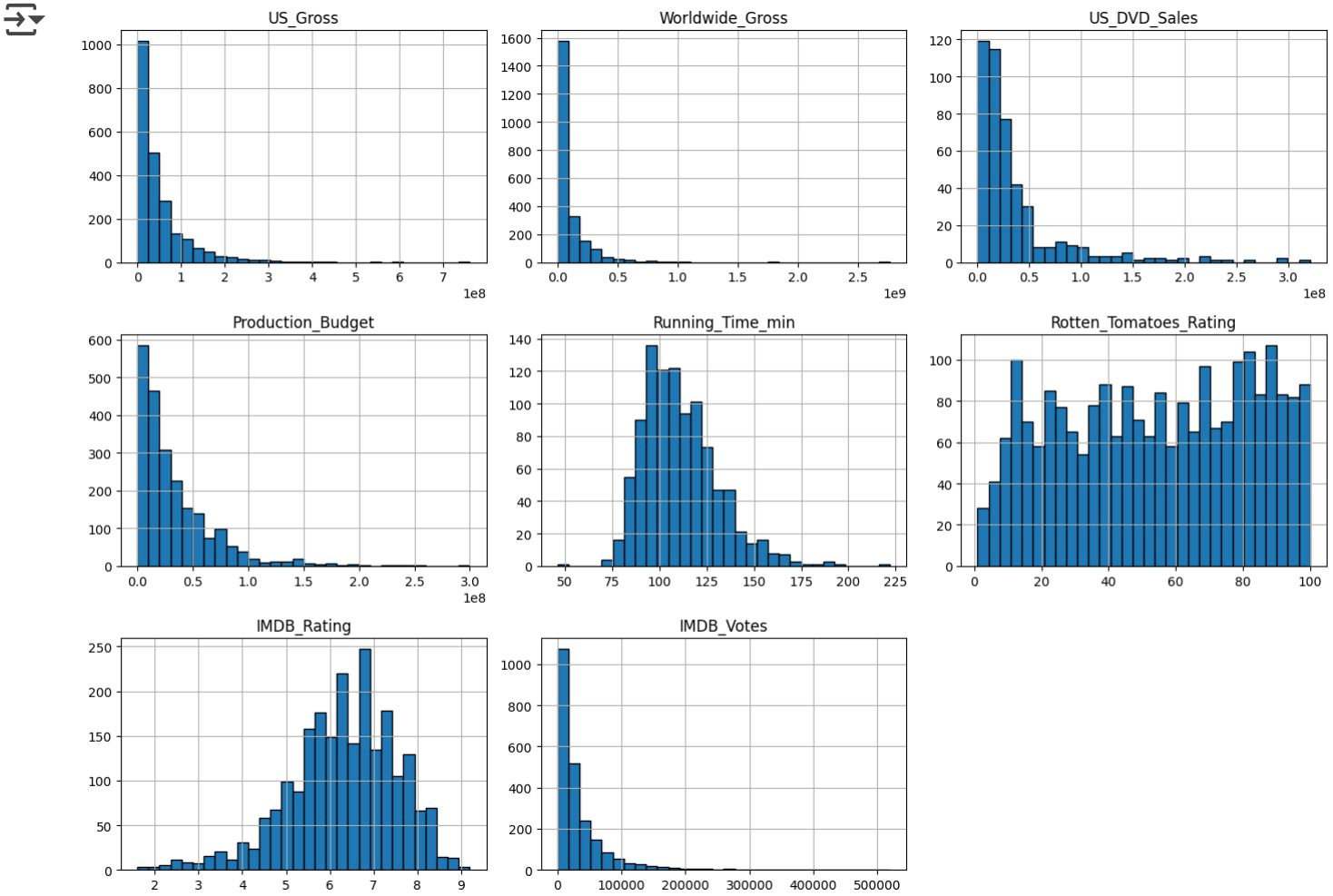
movies = data.movies()
movies = movies.dropna(subset=["IMDB_Rating", "IMDB_Votes", "Worldwide_Gross", "R
movies.head()
```

	Title	US_Gross	Worldwide_Gross	US_DVD_Sales	Production_Budget	Release
4	Slam	1009819.0	1087521.0	NaN	1000000.0	Oct 0
8	Pirates	1641825.0	6341825.0	NaN	40000000.0	Jul 0
9	Duel in the Sun	20400000.0	20400000.0	NaN	6000000.0	Dec 3
10	Tom Jones	37600000.0	37600000.0	NaN	1000000.0	Oct 0
11	Oliver!	37402877.0	37402877.0	NaN	10000000.0	Dec 1

If you simply call `hist()` method with a dataframe object, it identifies all the numeric columns and draw a histogram for each.

Q: draw all possible histograms of the movie dataframe. Adjust the size of the plots if needed.

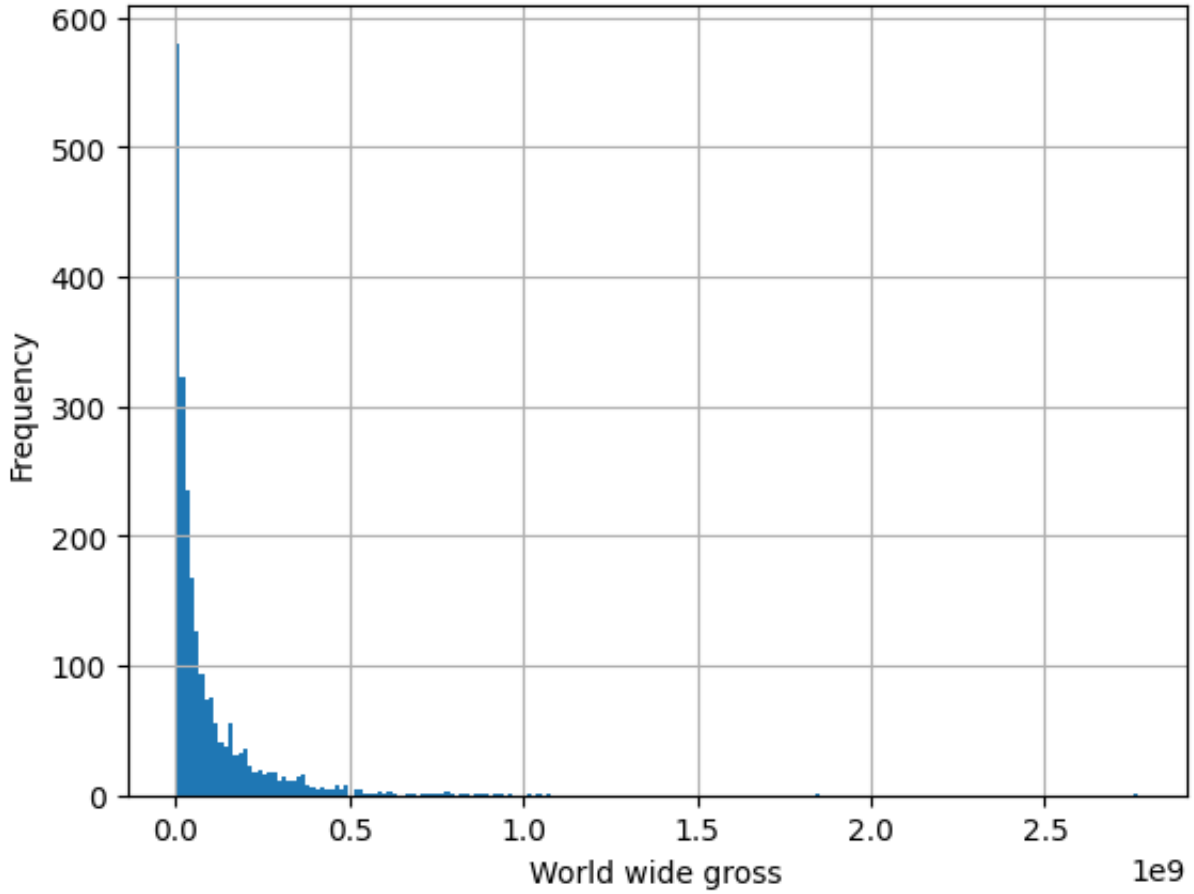
```
# YOUR SOLUTION HERE
movies.hist(figsize=(14, 10), bins=30, edgecolor='black')
plt.tight_layout()
plt.show()
```



As we can see, a majority of the columns are not normally distributed. In particular, if you look at the worldwide gross variable, you only see a couple of meaningful data from the histogram. Is this a problem of resolution? How about increasing the number of bins?

```
ax = movies["Worldwide_Gross"].hist(bins=200)
ax.set_xlabel("World wide gross")
ax.set_ylabel("Frequency")
```

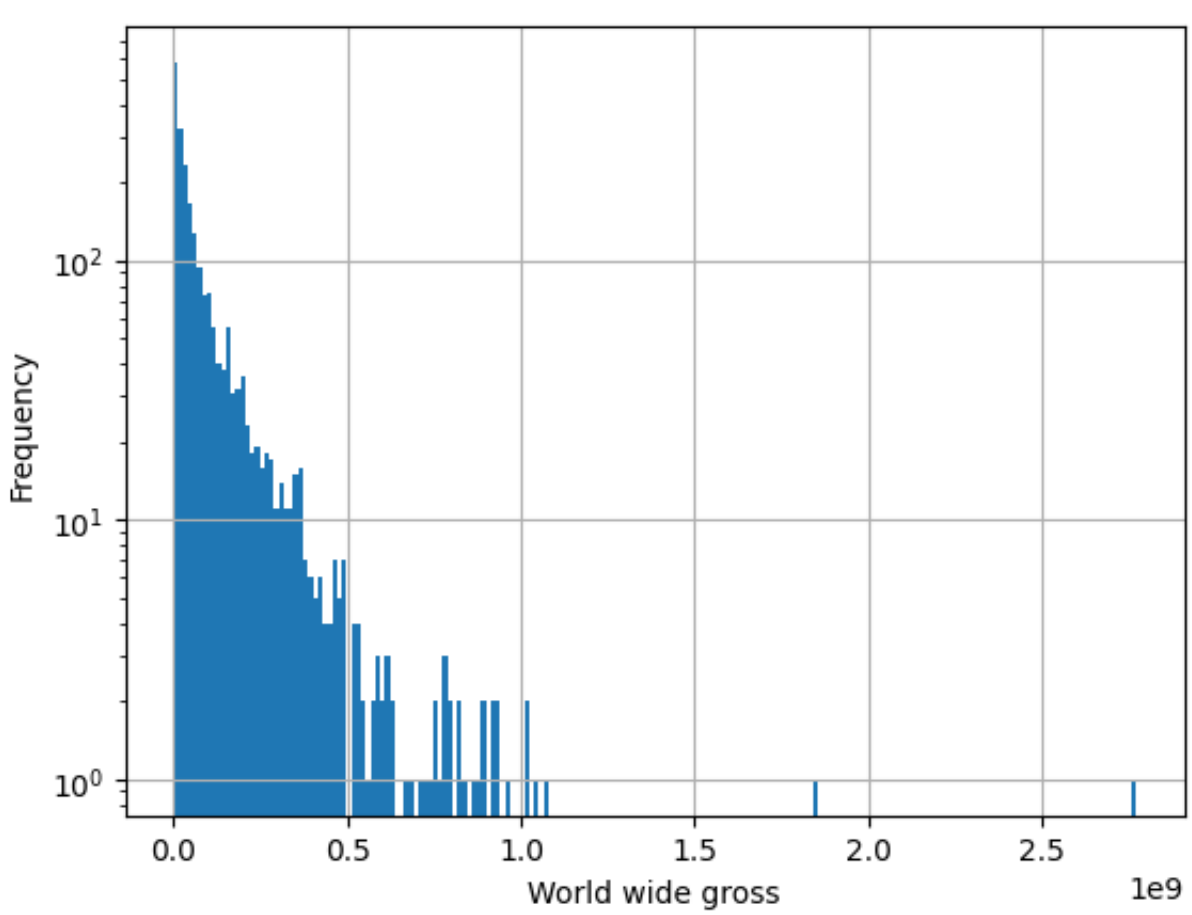
```
Text(0, 0.5, 'Frequency')
```



Maybe a bit more useful, but it doesn't tell anything about the data distribution above certain point. How about changing the vertical scale to logarithmic scale?

```
# YOUR SOLUTION HERE

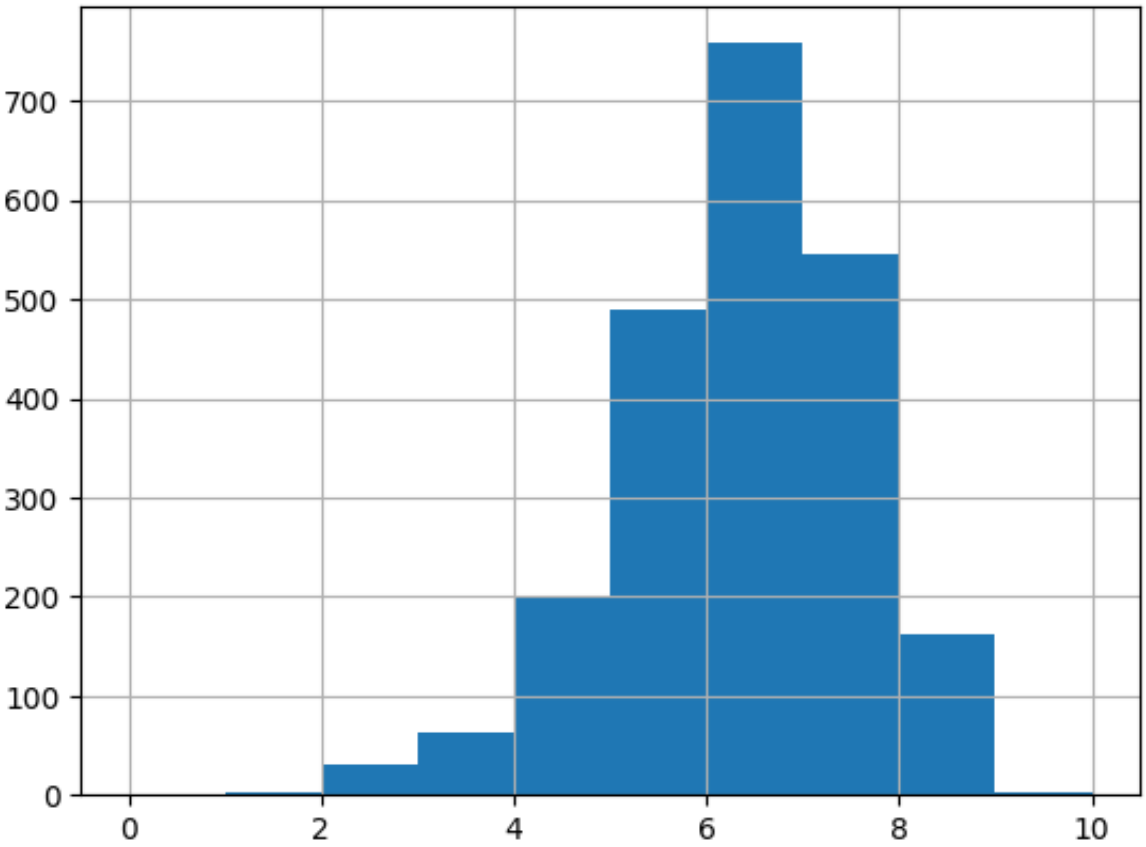
ax = movies["Worldwide_Gross"].hist(bins=200)
ax.set_yscale("log")
ax.set_xlabel("World wide gross")
ax.set_ylabel("Frequency")
plt.grid(True)
plt.show()
```



Now, let's try log-bin. Recall that when plotting histgrams we can specify the edges of bins through the `bins` parameter. For example, we can specify the edges of bins to [1, 2, 3, ... , 10] as follows.

```
movies["IMDB_Rating"].hist(bins=range(0,11))
```

```
<Axes: >
```



Here, we can specify the edges of bins in a similar way. Instead of specifying on the linear scale, we do it on the log space. Some useful resources:

- Google query: python log-bin
- numpy.logspace
- numpy.linspace vs numpy.logspace

Hint: since $10^{\text{start}} = \min(\text{Worldwide_Gross})$, $\text{start} = \log_{10}(\min(\text{Worldwide_Gross}))$

```
min(movies["Worldwide_Gross"])
```

```
0.0
```

Because there seems to be movie(s) that made $0, and because log(0) is undefined & log(1) = 0, let's add 1 to the variable.

```
movies["Worldwide_Gross"] = movies["Worldwide_Gross"]+1.0
```

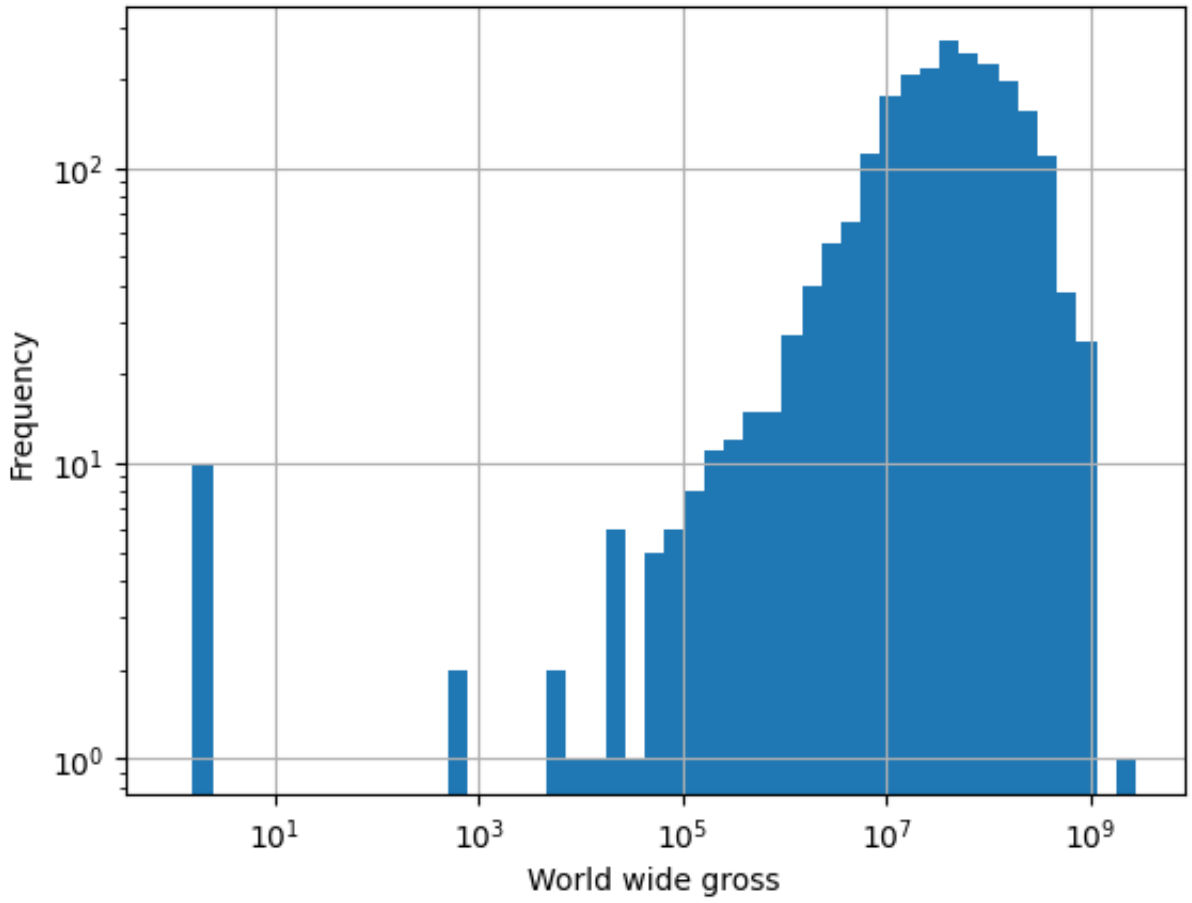
```
# TODO: specify the edges of bins using np.logspace

# YOUR SOLUTION HERE
bins = np.logspace(
    np.log10(movies["Worldwide_Gross"].min()),
    np.log10(movies["Worldwide_Gross"].max()),
    num=50
)
```

Now we can plot a histgram with log-bin. Set both axis to be log-scale.

```
ax = (movies["Worldwide_Gross"]+1.0).hist(bins=bins)
ax.set_yscale('log')
ax.set_xscale('log')
ax.set_xlabel("World wide gross")
ax.set_ylabel("Frequency")
```

```
Text(0, 0.5, 'Frequency')
```



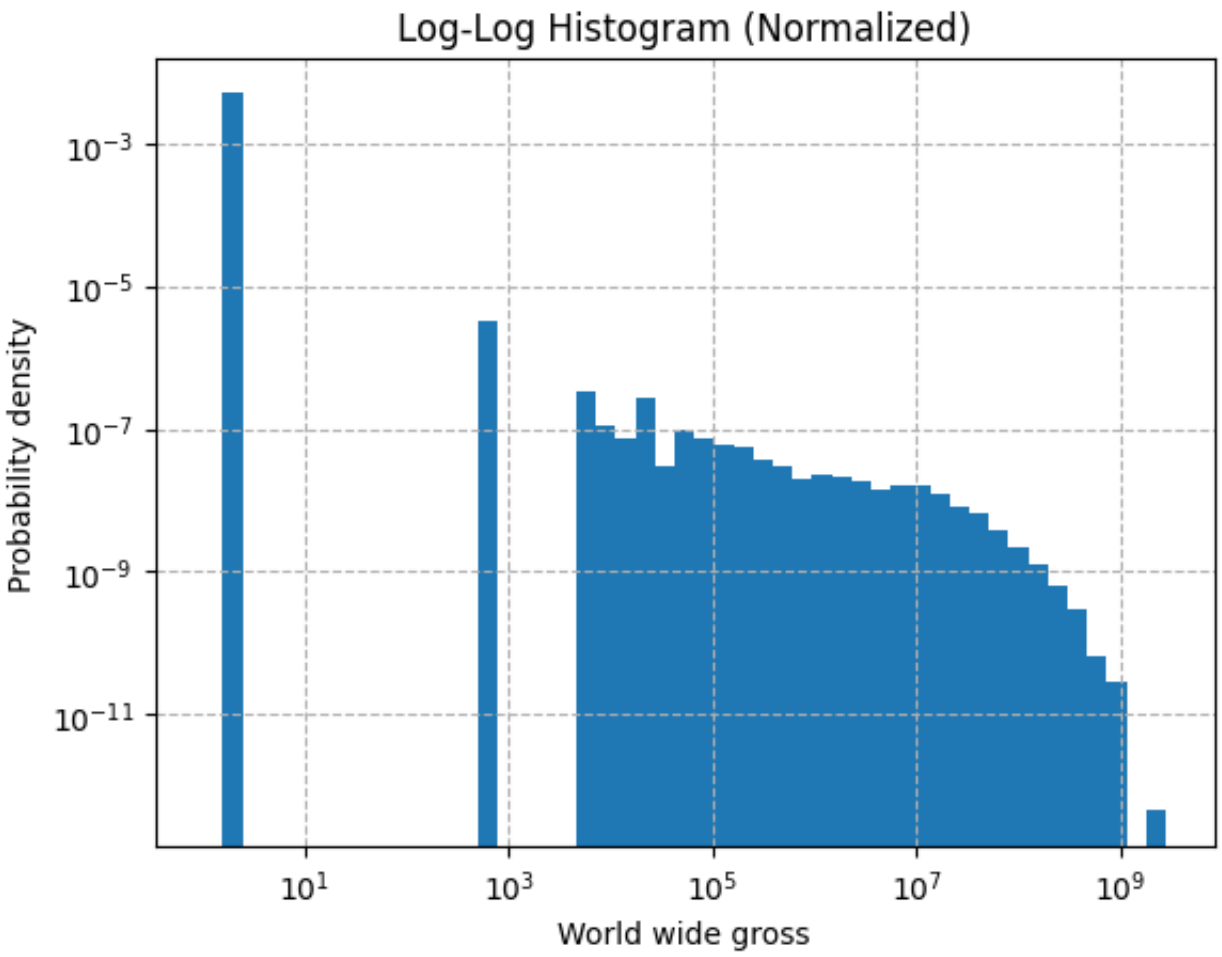
What is going on? Is this the right plot?

Q: explain and fix

This is not the correct plot. The issue is that we're plotting a histogram on a log-log scale using log-binned edges, but the `density=True` parameter wasn't specified. Without it, the plot shows raw counts per bin, and the bin widths are unequal due to the log spacing. This makes the bars misleading—taller bars appear even if they represent fewer data points.

To fix it, we should add `density=True` to normalize by bin width, and also use `+1.0` to avoid log(0).

```
# YOUR SOLUTION HERE
plt.hist(movies["Worldwide_Gross"] + 1.0, bins=bins, density=True)
plt.xscale('log')
plt.yscale('log')
plt.xlabel("World wide gross")
plt.ylabel("Probability density")
plt.grid(True, which='both', ls='--')
plt.title("Log-Log Histogram (Normalized)")
plt.show()
```



Interactive exploration of log-binning

Let's put the linear and log-binned histogram side by side to explore how the number of bins affects the log-binned histogram visualization. The slider below allows you to experiment with different numbers of bins to see how it impacts the visualization of the heavy-tailed distribution.

```
from ipywidgets import interact
import matplotlib.pyplot as plt
import numpy as np

@interact(n_bins=(5, 100, 5))
def plot_log_histogram(n_bins):
    """
    Interactive visualization to explore how bin number affects log-binned histogra

    Parameters:
    n_bins: Number of bins for the histogram
    """

    # Create linear-spaced bins
    linear_bins = np.linspace(movies["Worldwide_Gross"].min(), movies["Worldwide_Gr

    # Create log-spaced bins
    log_bins = np.logspace(np.log10(movies["Worldwide_Gross"].min()),
                           np.log10(movies["Worldwide_Gross"].max()),
                           n_bins)

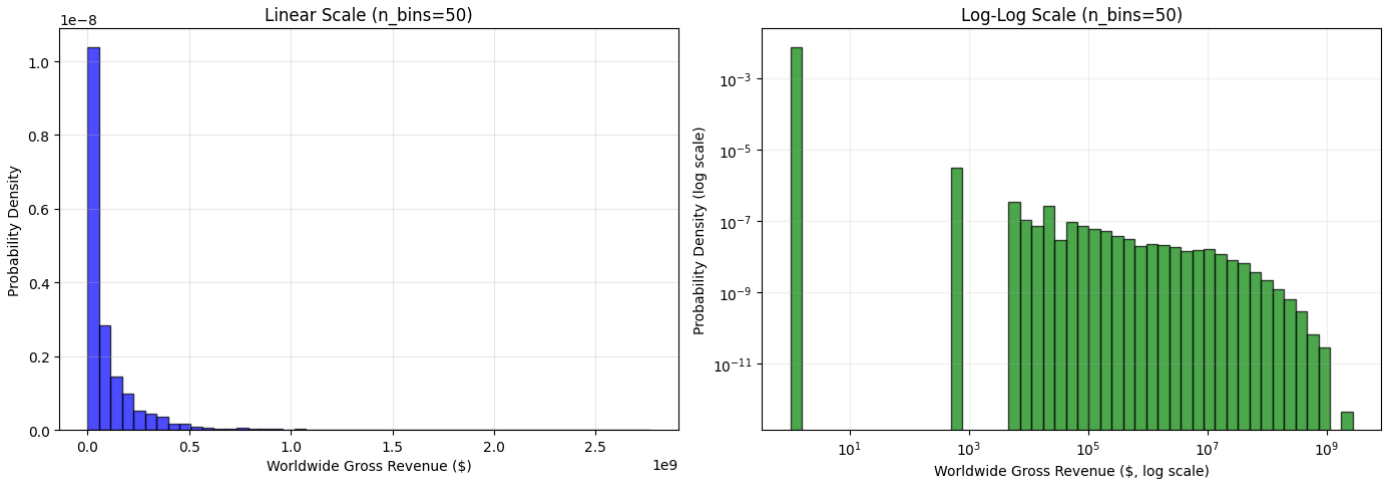
    # Create figure with two subplots
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    # Left plot: Linear scale
    ax1.hist(movies["Worldwide_Gross"], bins=linear_bins, density=True, alpha=0.7,
    ax1.set_xlabel("Worldwide Gross Revenue ($)")
    ax1.set_ylabel("Probability Density")
    ax1.set_title(f"Linear Scale (n_bins={n_bins})")
    ax1.grid(True, alpha=0.3)

    # Right plot: Log-log scale
    ax2.hist(movies["Worldwide_Gross"], bins=log_bins, density=True, alpha=0.7, col
    ax2.set_xscale('log')
    ax2.set_yscale('log')
    ax2.set_xlabel("Worldwide Gross Revenue ($, log scale)")
    ax2.set_ylabel("Probability Density (log scale)")
    ax2.set_title(f"Log-Log Scale (n_bins={n_bins})")
    ax2.grid(True, which="both", ls="-", alpha=0.2)

    plt.tight_layout()
    plt.show()
```

n_bins 50



CCDF

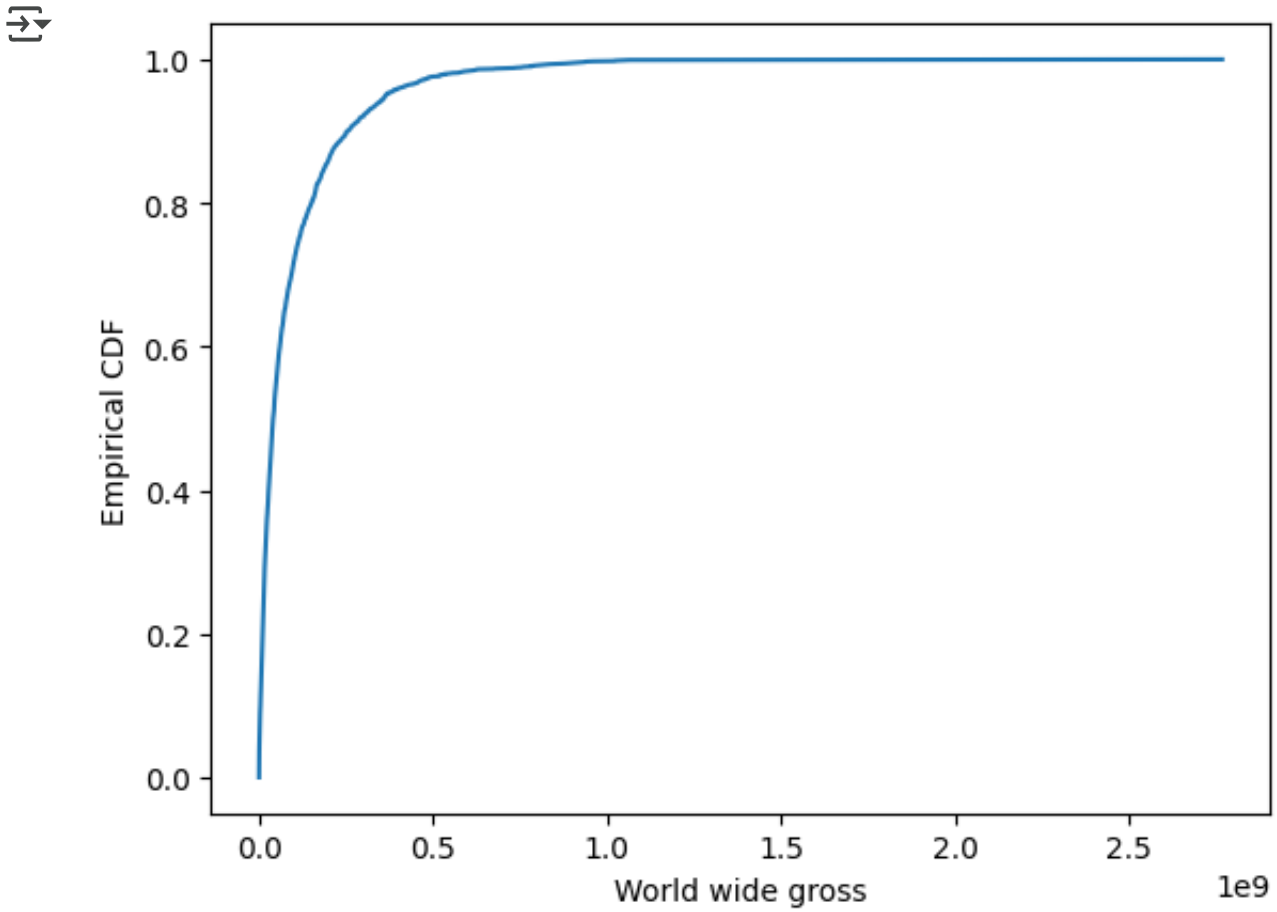
The cumulative distribution function $F_X(x)$ at x is defined by

$$F_X(x) = P(X \leq x),$$

which is, in other words, the probability that X takes a value less than or equal to x. When empirically calculated (empirical CDF), $F_X(x)$ is the fraction of data points that are less than or equal to x. CDF allows us to examine any percentile of the data distribution and is also useful for comparing distributions.

However, when the data spans multiple orders of magnitude, CDF may not be useful. Let's try.

```
gross_sorted = movies.Worldwide_Gross.sort_values()
N = len(gross_sorted)
Y = np.linspace(1/N, 1, num=N)
plt.xlabel("World wide gross")
plt.ylabel("Empirical CDF")
_ = plt.plot(gross_sorted, Y)
```



Although the movies that are interesting are those with large worldwide gross, we don't see any details about their distribution as they are all close to 1. In other words, CDF sucks at revealing the details of the tail.

CCDF is a nice alternative to examine distributions with heavy tails. The idea is same as CDF, but the direction of aggregation is opposite. Because we are starting from the largest value, it can reveal the details of those large values (tail).

CCDF is defined as follows:

$$\bar{F}_X(x) = P(X > x)$$

And thus,

$$\bar{F}_X(x) = P(X > x) = 1 - F_X(x)$$

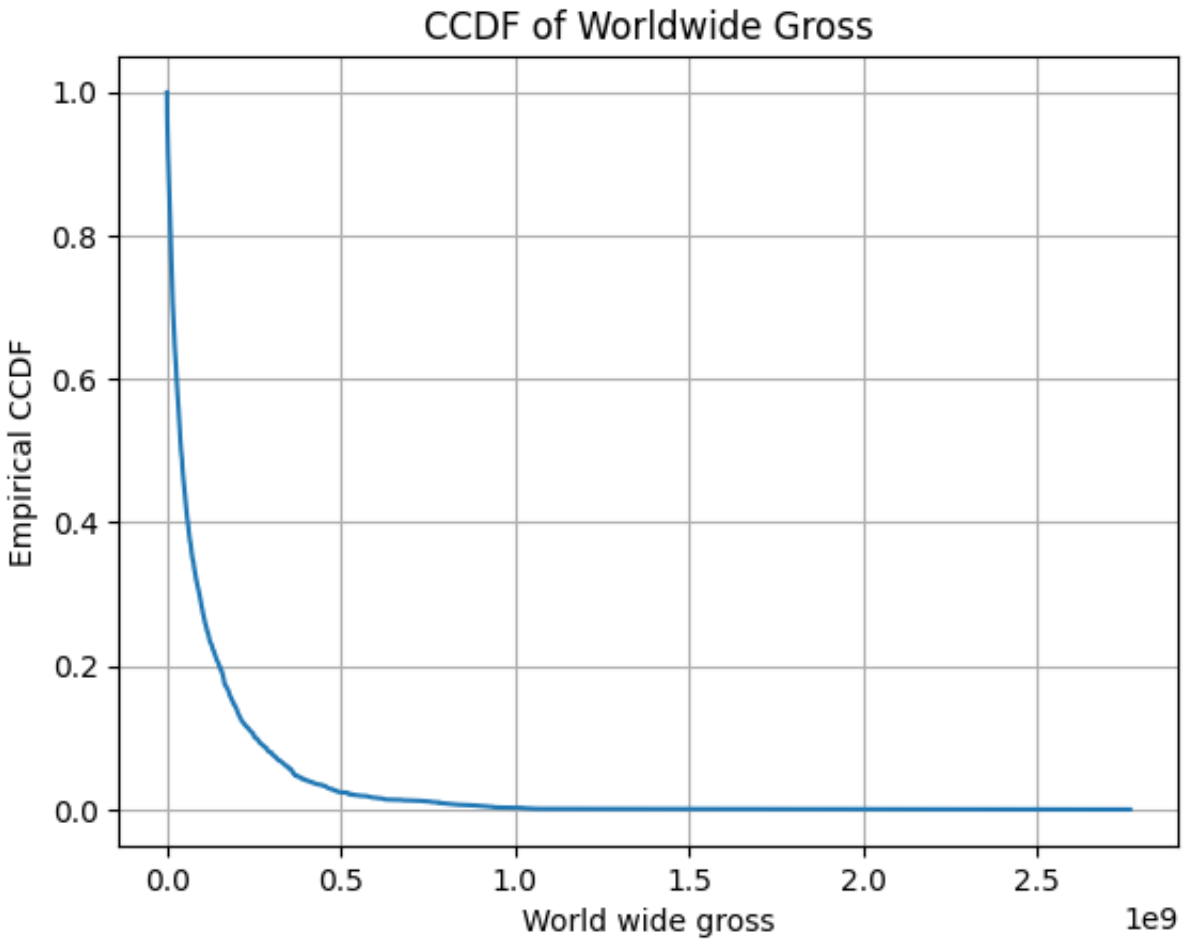
In other words, we can use CDF to calculate CCDF.

Q: draw CCDF using the CDF code above.

```
# YOUR SOLUTION HERE

sorted_gross = np.sort(movies["Worldwide_Gross"])
cdf = np.arange(1, len(sorted_gross)+1) / len(sorted_gross)
ccdf = 1 - cdf

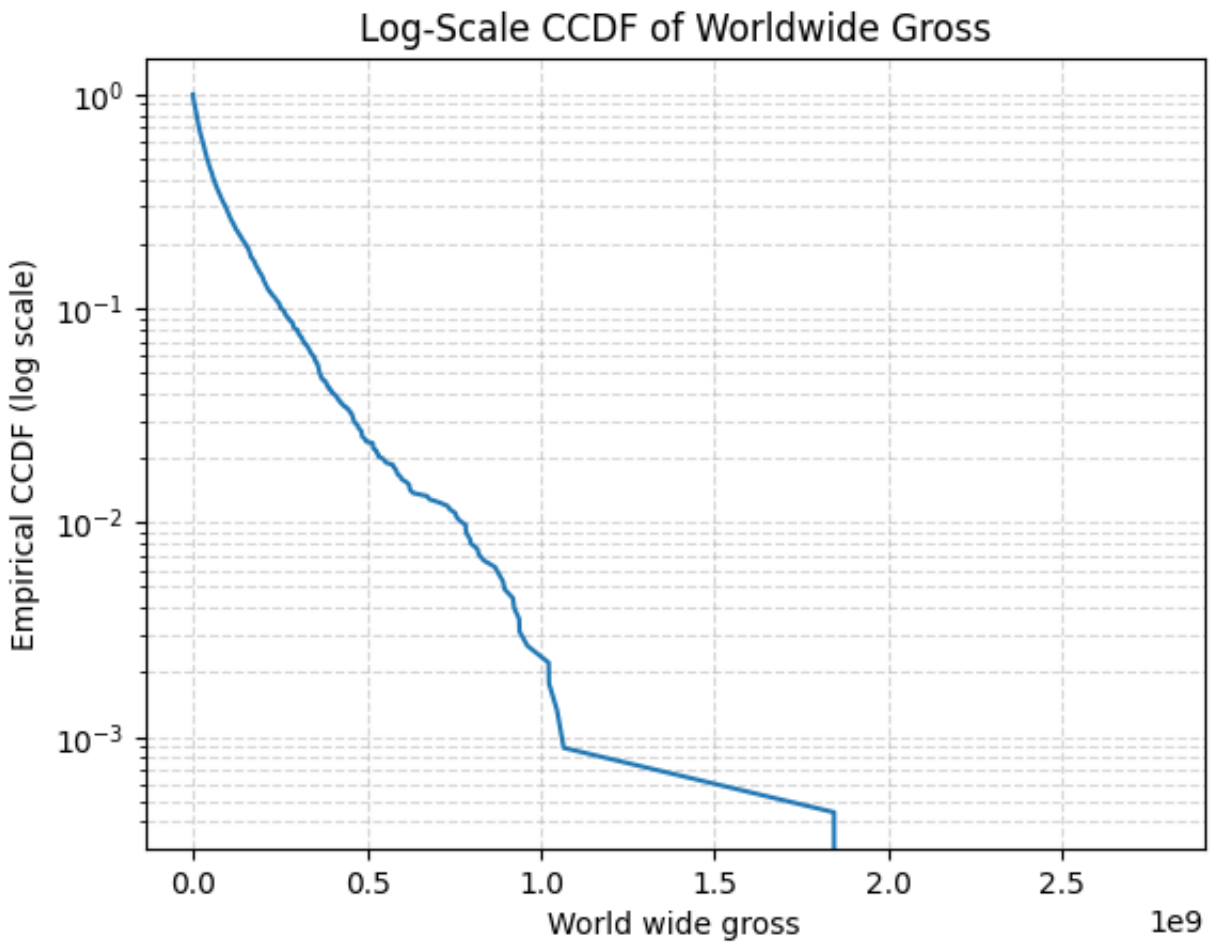
plt.plot(sorted_gross, ccdf)
plt.xlabel("World wide gross")
plt.ylabel("Empirical CCDF")
plt.title("CCDF of Worldwide Gross")
plt.grid(True)
plt.show()
```



How about making the y axis in log scale?

```
# YOUR SOLUTION HERE

plt.plot(sorted_gross, ccdf)
plt.yscale("log")
plt.xlabel("World wide gross")
plt.ylabel("Empirical CCDF (log scale)")
plt.title("Log-Scale CCDF of Worldwide Gross")
plt.grid(True, which='both', linestyle='--', alpha=0.5)
plt.show()
```



Although this is technically the correct CCDF plot, there is a very subtle issue. Do you see the vertical line at the rightmost side of the CCDF plot? To understand what's going on, let's look at the Y values of this plot. We used 1 - CDF to calculate CCDF. So,

```
1 - Y
```

```
array([9.99556738e-01, 9.99113475e-01, 9.98670213e-01, ...,
       8.86524823e-04, 4.43262411e-04, 0.00000000e+00])
```

What happens when we take the log of these values?

```
np.log(1-Y)
```

```
/tmp/ipython-input-20-1767632406.py:1: RuntimeWarning: divide by zero encounte
  np.log(1-Y)
array([-4.43360681e-04, -8.86918018e-04, -1.33067219e-03, ...,
       -7.02820143e+00, -7.72134861e+00,            -inf])
```

Because the last value of 1 - Y is 0.0, we got `-inf` as the log value. That means, the largest value's (let's say x) coordinate in our CCDF plot will be $(x, -inf)$ if we use a log scale for our y-axis. And thus we will not be able to see it in the plot. This occurs because we are drawing CDF in a simplified way. In reality, ECDF and ECCDF are step functions and this shouldn't matter. However, because we are drawing a line between the points, we are getting this issue.

This is somewhat problematic because the largest value in our dataset can be quite important and therefore we want to see it in the plot!

This is why, in practice, we sometimes use "incorrect" version of CCDF. We can consider $\bar{F}_X(x)$ as a "flipped" version of CDF.

$$\bar{F}_X(x) = P(X \geq x)$$

instead of

$$\bar{F}_X(x) = P(X > x)$$

In doing so, we can see the largest value in the data in our CCDF plot. We can also draw the correct version of CCDF, but this quick-and-dirty version is often easier and good enough to show what we want to show.

A simple way is just to define the y coordinates as follows:

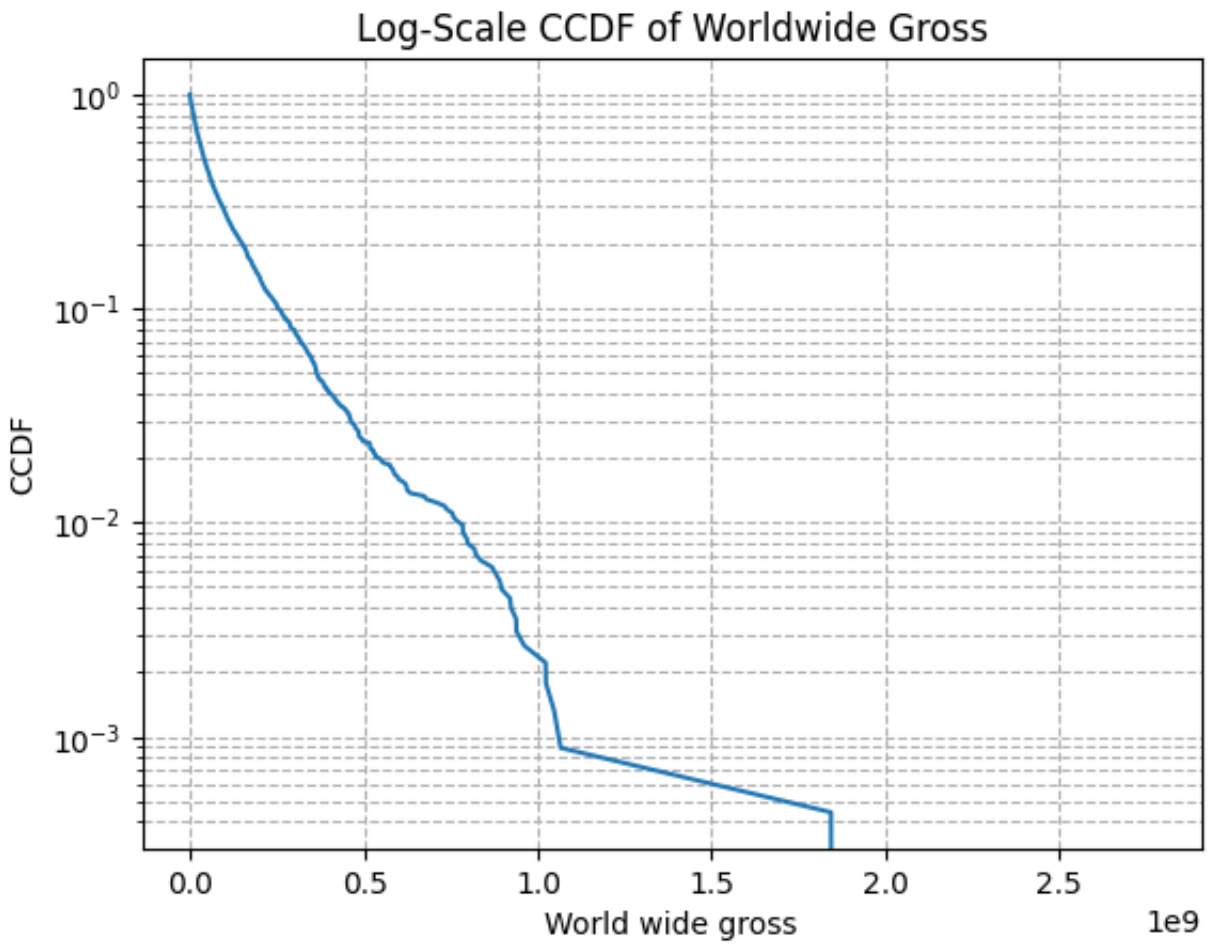
```
Y = np.linspace( 1.0, 1/N, num=N)
```

Q: Draw a CCDF of worldwide gross data. Use log scale for y-axis.

```
# YOUR SOLUTION HERE

sorted_gross = np.sort(movies["Worldwide_Gross"])
cdf = np.arange(1, len(sorted_gross)+1) / len(sorted_gross)
ccdf = 1 - cdf

plt.plot(sorted_gross, ccdf)
plt.yscale("log")
plt.xlabel("World wide gross")
plt.ylabel("CCDF")
plt.title("Log-Scale CCDF of Worldwide Gross")
plt.grid(True, which='both', linestyle='--')
plt.show()
```



A straight line in semilog scale means exponential decay (cf. a straight line in log-log scale means power-law decay). So it seems like the amount of money a movie makes across the world follows *roughly* an exponential distribution, while there are some outliers that make insane amount of money.

Q: Which is the most successful movie in our dataset?

You can use the following

- `idxmax()` : https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.idxmax.html
- `loc` : https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.loc.html

```
# YOUR SOLUTION HERE

most_successful = movies.loc[movies["Worldwide_Gross"].idxmax()]
print(most_successful)
```

```
Title                                 Avatar
US_Gross                          760167650.0
Worldwide_Gross                  2767891500.0
US_DVD_Sales                      146153933.0
Production_Budget                 237000000.0
Release_Date                      Dec 18 2009
MPAA_Rating                             PG-13
Running_Time_min                          NaN
Distributor                 20th Century Fox
Source                  Original Screenplay
Major_Genre                            Action
Creative_Type                  Science Fiction
Director                         James Cameron
Rotten_Tomatoes_Rating                    83.0
IMDB_Rating                                8.3
IMDB_Votes                            261439.0
Name: 1234, dtype: object
```

Practical Summary: When to Use Each Technique

Ok! After working through this lab, you should now understand when and how to use logarithmic scales and related techniques. Here's a practical guide for real-world applications:

When to Use Log Scale

Use log scale when:

- Comparing ratios: Ratios need symmetric representation (e.g., 10:1 vs 1:10)
- Data spans multiple orders of magnitude: Values range from 1 to 1,000,000+
- Multiplicative relationships: Growth rates, compound interest, population growth
- Visualizing percentage changes: Stock prices, economic indicators
- Heavy-tailed distributions: Income, city sizes, word frequencies

Common applications:

- Financial data (stock prices, market caps)
- Scientific measurements (earthquake magnitudes, sound intensity)
- Web analytics (page views, user engagement)
- Biological data (population sizes, gene expression)

When to Use Log-binning

Use log-binning when:

- Creating histograms for heavy-tailed distributions
- Linear bins would result in most data falling into the first few bins
- You need to see the structure across the entire range of values
- The data follows a power law, log-normal, or some heavy-tailed distribution

Key considerations:

- Normalize by bin width (use `density=True`). Otherwise, you may get very misleading results.
- Experiment with different numbers of bins to find the right balance
- Try both linear and log-scale views for comparison

When to Use CCDF (Complementary CDF)

Use CCDF when:

- Analyzing tail behavior of distributions
- Comparing multiple heavy-tailed distributions
- Identifying outliers or extreme events

Advantages over regular/log-binned histograms:

- No binning artifacts
- Full use of the data
- Easier to compare multiple distributions
- Clear visualization of rare events

Quick Decision Guide

1. "**My data values vary across many orders of magnitude, like from 10 to 10,000,000**" → try log-scale and log-binning
2. "**I'm comparing growth rates or ratios**" → try log-scale
3. "**My histogram has one tall bar and everything else is tiny**" → try log-binning

Common Pitfalls to Avoid

1. **Pay attention to zeros and negative values** in your data
2. **Don't forget to normalize histograms** when your bins are not uniform (e.g., log-binning)
3. **Always label axes clearly** to indicate log scale usage and binning
4. **Consider your audience** - add explanatory notes if they may not be familiar with log scales and binning

Python Quick Reference

```
# Log scale for current plot
plt.yscale('log')     # using matplotlib.pyplot
ax.set_yscale('log')  # using ax object

# Log-binning
bins = np.logspace(np.log10(data.min()), np.log10(data.max()), n_bins)
plt.hist(data, bins=bins, density=True)   # density=True is important!

# Quick and dirty CCDF
data_sorted = np.sort(data)
ccdf = np.linspace(1, 1/len(data), len(data))
plt.loglog(data_sorted, ccdf)  # for log-log scale
```